applied sciences

MDPI

Article

A Carbon-Efficient Framework for Deep Learning Workloads on GPU Clusters

Dong-Ki Kang [1] and Yong-Hyuk Moon [2,*]

[1] Division of Electronic and Information, Department of Computer Engineering, Jeonbuk National University, Jeonju 54896, Republic of Korea; dongkikang@jbnu.ac.kr
[2] Department of Computer Engineering, Sungshin Women's University, Seoul 02844, Republic of Korea
* Correspondence: yhmoon@sungshin.ac.kr; Tel.: +82-02-920-7528

Abstract

The explosive growth of artificial intelligence (AI) services has led to massive scaling of GPU computing clusters, causing sharp rises in power consumption and carbon emissions. Although hardware-level accelerator enhancements and deep neural network (DNN) model compression techniques can improve power efficiency, they often encounter deployment barriers and risks of accuracy loss in practice. To address these issues without altering hardware or model architectures, we propose a novel Carbon-Aware Resource Management (CA-RM) framework for GPU clusters. In order to minimize the carbon emission, the CA-RM framework dynamically adjusts energy usage by combining real-time GPU core frequency scaling with intelligent workload placement, aligning computation with the temporal availability of renewable generation. We introduce a new metric, performance-per-carbon (PPC), and develop three optimization formulations: carbon-constrained, performance-constrained, and PPC-driven objectives that simultaneously respect DNN model training deadlines, inference latency requirements, and carbon emission budgets. Through extensive simulations using real-world renewable energy traces and profiling data collected from NVIDIA RTX4090 GPU running representative DNN workloads, we show that the CA-RM framework substantially reduces carbon emission while satisfying service-level agreement (SLA) targets across a wide range of workload characteristics. Through experimental evaluation, we verify that the proposed CA-RM framework achieves approximately 35% carbon reduction on average, compared to competing approaches, while still ensuring acceptable processing performance across diverse workload behaviors.

Keywords: carbon emission; deep neural network; frequency scaling; renewable energy; training; inference

Check for updates

Academic Editor: Douglas O'Shaughnessy

Received: 29 November 2025
Revised: 28 December 2025
Accepted: 30 December 2025
Published: 7 January 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Recent explosive growth in artificial intelligence (AI)-driven applications and services has led to a rapid expansion of graphics processing unit (GPU) clusters, the essential infrastructure for processing large-scale AI workloads [1]. Consequently, the energy consumption of GPU clusters has also increased at an exponential rate. Although GPU device architectures continue to improve in terms of performance-per-watt (PPW), the continuous growth in computational demand has far outpaced these efficiency gains, resulting in a continuous rise in the total energy consumed by modern GPU clusters. For example, NVIDIA's Blackwell-based GPU architecture such as RTX5090, released in late 2024, features a total graphics power (TGP) of approximately 575W [2]. This level of consumption

is roughly 1.5 times higher than that of recent high-performance CPUs such as the Intel Xeon Platinum 8480+ (350W TDP) [3] and AMD EPYC 9654 (360W TDP) [4]. According to [5], global data center electricity consumption is expected to exceed 945TWh by 2030, and this level corresponds to roughly 3% of global electricity demand. In particular, the energy demand of AI-related processing will be approaching 35% of overall data center energy consumption in 2030 [6].

Rising energy consumption leads to an increase in carbon emissions. According to a recent global assessment report, the average carbon intensity of electricity generation in 2023 was approximately 480 gCO_2e/Wh, indicating that each additional unit of electricity consumption proportionally contributes to carbon emissions [7]. This continued growth is critical because CO_2 emissions directly accelerate global warming. Empirical climate–carbon cycle analyses suggest that every additional 1000 $GtCO_2$ emitted is associated with roughly 0.45 °C of long-term global temperature increase [8]. To address these problems, the Paris Agreement (2015) established a global climate framework aiming to keep the rise in global mean temperature well below 2 °C above pre-industrial levels while pursuing efforts to limit warming to 1.5 °C [9]. Under this framework, each Party must submit a Nationally Determined Contribution (NDC), a national emissions-reduction pathway and carbon budget, and is subject to increasing economic, political, and regulatory pressure if it fails to meet these commitments.

Given the constrained global carbon budgets and the rapid growth of AI workloads, reducing the carbon emissions associated with GPU-based AI processing has become essential. Conventional approaches for lowering carbon emissions in GPU clusters generally fall into three categories: (1) reducing the energy consumption of accelerator hardware through improved architecture-level design, (2) modifying the structure of deep neural network (DNN) models to reduce computational and energy demands, and (3) conducting scheduling-based energy- and carbon-aware GPU cluster management. From the perspective of architecture-level design, Google reports that its Cloud TPU v4 delivers approximately 2.7× higher performance-per-watt (PPW) compared to TPU v3, with typical chip power in the range of 200–300 W [10]. Similarly, FuriosaAI's NPUs (Warboy and Renegade/RNGD) demonstrate over 2× higher inference PPW and more than 60% greater energy efficiency than comparable NVIDIA GPU-based systems for LLM inference—corresponding to roughly 40–50% energy savings for equal throughput [11]. From the perspective of NN model optimization, techniques such as model pruning, quantization, and knowledge distillation can significantly reduce energy consumption while maintaining acceptable accuracy. For example, training the LLaMA-7B model requires approximately 36 MWh of electricity, whereas LLaMA-65B requires around 449 MWh, meaning the 7B model consumes only about 8% of the training energy while still achieving sufficient performance for many downstream tasks [12]. From the perspective of energy- and carbon-aware scheduling, ref. [13] proposes an energy-optimized cluster management scheme for clusters containing heterogeneous GPU devices. They design the GPU utilization prediction method and GPU heterogeneity-aware energy consumption model. Based on the Alibaba trace, they validated that their work improves energy saving by about 35%. Ref. [14] proposes a carbon-efficient machine learning job scheduler that enables the job moving to greener hours so as to decrease the carbon emission. Their proposed approach improves the carbon emission reduction by about 40% with negligible performance degradation.

However, the strategies described above remain challenging in practice. Hardware design and fabrication for new AI accelerators typically require long development cycles, substantial capital investment, and specialized manufacturing processes. As a result, newly developed devices cannot be immediately or universally deployed across existing GPU-based infrastructures, which often operate under strict compatibility and reliabil-

ity constraints [15]. Similarly, DNN model compression techniques reduce parameter counts and representational capacity, and therefore may introduce non-trivial accuracy degradation, particularly when applied with aggressive compression ratios or on complex, fine-grained downstream tasks [16,17]. Thus, compression must be carefully balanced against application-specific accuracy, robustness, and latency requirements rather than treated as a universally applicable solution for carbon or energy reduction. Prior studies on energy- and carbon-aware scheduling for GPU clusters do not explicitly consider both training and inference workloads under time-varying renewable energy generation. Moreover, their proposed formulations typically lack sophisticated constraints for enforcing predefined service-level agreements (SLAs) across heterogeneous user requests. As a result, these approaches are not readily applicable to general-purpose GPU-based AI data centers.

To address these challenges, in this paper, we propose a novel Carbon-Aware Resource Management (CA-RM) framework for AI workload processing on GPU clusters, focusing on the carbon emission factor and maximizing the utilization of low-carbon renewable electricity sources, without hardware re-design and DNN model compression. The carbon emission factor represents the amount of carbon dioxide emitted per unit of electricity generated. It is worth noting that even the same GPU energy usage can lead to very different carbon footprints depending on whether the marginal electricity comes from renewable (low-carbon emission factor) sources like wind farm and solar photovoltaics or from fossil (high-carbon emission factor) sources such as coal, oil, or natural gas [18]. Table 1 shows the carbon emission factors for each energy source.

Table 1. Carbon emission factors (gCO_2e/Wh) [18].

Coal	Wind	Solar Photovoltaics	Geothermal	Diesel	Natural Gas (LNG)
0.96	0.034	0.05	0.038	0.778	0.611

In our CA-RM framework, we utilize two methodologies for the reduction of carbon emission: the GPU core frequency scaling and computing worker allocation. The GPU core frequency scaling dynamically tunes the GPU core operating frequency for inference service processing in real time, which directly adjusts GPU power draw, processing performance, and resulting energy consumption [19]. Note that GPU core frequency scaling does not require any non-negligible control overheads and most modern GPU devices deploy the scaling instruction set within themselves. Based on these findings, our CA-RM framework uses frequency scaling as an energy-shifting lever; when renewable generation is scarce, we lower the GPU core clock to reduce instantaneous consumption and thus minimize brown-energy usage; when renewable generation is abundant, we raise the clock to increase consumption and maximize renewable utilization. This allows the cluster's energy demand to better track renewable supply over time. In the computing worker allocation process, we decide how many computing workers are assigned to DNN model training jobs and when they begin to process the allocated jobs. When renewable generation is low, we reduce the number of parallel workers or postpone the training jobs to later time slots and when renewable generation is high, we increase the number of workers and immediately process the assigned training jobs. Note that the training jobs are batch workload, not like inference tasks as transactional workload, and they are deferrable within the predefined deadline. Combined to GPU core frequency scaling for inference service processing, the computing worker allocation enables the adaptive energy-shifting corresponding to renewable generation behaviors.

This paper makes the following contributions.

- We propose a Carbon-Aware Resource Management (CA-RM) framework for GPU clusters that process AI workloads. The framework explicitly targets minimizing operational carbon emissions by maximizing the utilization of low-carbon renewable electricity, while still meeting performance objectives such as training deadline and inference latency bound.
- To quantify the carbon usage efficiency, we introduce the novel metric called performance-per-carbon (PPC) for both DNN model training jobs and inference tasks. By using the PPC metric, we can compare our proposed CA-RM framework to other methods quantitatively and verify the practicalness of our work. Furthermore, we deploy the carbon budget constraints to our formulation so as to achieve the acceptable carbon emission under various workload patterns.
- We utilize both real-time GPU core frequency scaling and computing worker allocation approaches that adapt GPU workers' power draw and energy use to temporal changes in renewable generation. In order to minimize the carbon emission while guaranteeing the performance requirement of AI workloads, we present the sophisticated mathematical formulation that explicitly controls the carbon emission, DNN model training time, and inference service latency. In particular, we design carbon-constrained, performance-constrained, and PPC-driven optimization approaches in our CA-RM framework so as to achieve various targets under the various problems described.
- To demonstrate the performance of our proposed CA-RM framework, we utilize the real renewable generation traces of wind farms and solar photovoltaics, and retrieve the profiling data from an NVIDIA RTX4090 device with well-known DNN models such as vgg16, inceptionv4, resnet152, and stochasticdepth101. Based on the raw data, we conduct an extensive simulation experiment and validate that our CA-RM framework is functional given various workload patterns.

Section 2 depicts the structure of our proposed CA-RM framework. In Section 3, we present the system model including performance, energy consumption, and carbon emission models for DNN model training and inference. Section 4 formulates the optimization problems involving carbon emission constraints, performance constraints, PPC-aware objectives, and the resource management algorithm. Section 5 shows the raw profiling data of the RTX4090 device with DNN models and various evaluation results of our CA-RM framework over its competitors. We summarize and conclude our work in Section 6.

2. Proposed Framework Structure

Our proposed Carbon-Aware Resource Management (CA-RM) framework is illustrated in Figure 1.

(1) Electricity Market: The electricity market provides the fossil-based grid energy, denoted as $r^c(t)$. This component enables the CA-RM framework to purchase fossil energy when the renewable supply is insufficient. The renewable generation signal is periodically updated and incorporated into the CA-RM framework decision-making process to achieve carbon-efficient operation.

(2) Renewable Energy Generators: This module produces green energy from renewable generation sources such as wind farms denoted as $r^w(t)$ and solar photovoltaics $r^{pv}(t)$. This energy is first directed to satisfy the operational demand of the power delivery system (PDS) and is then used to supply the training cluster and the inference sub clusters. Corresponding to dynamic $r^w(t)$ and $r^{pv}(t)$, the CA-RM framework decides the amount of fossil energy to be purchased, $r^c(t)$, and supplies the required power to each cluster.

(3) Training Cluster: The training cluster executes multiple DNN model training jobs that are typically energy-intensive and long-duration. Its operation is coordinated by the

parallel worker allocation plan $n(t)$ and time slot assignment plan $x(t)$. The CA-RM framework may defer or slow down training jobs during high-carbon periods to align energy consumption with low-carbon availability.

(4) Inference Cluster: The inference cluster consists of multiple sub-clusters $(1, 2, \ldots, H)$ designed to handle real-time inference requests. Because inference workloads are latency-sensitive, this module prioritizes responsiveness while still allowing for a limited frequency scaling plan $f(t)$. The CA-RM framework dynamically assigns inference tasks among sub-clusters by involving DNN models. Through sub-cluster partitioning, the inference infrastructure achieves both scalability and fine-grained energy control.

(5) Carbon-Aware Resource Management Framework: The Carbon-Aware Resource Management (CA-RM) framework integrates DNN workload shifting and carbon emission management. It consists of two functional components: Tuning Manager and Optimization. The Tuning Manager is responsible for GPU core frequency scaling, parallel worker allocation, and time slot assignment. It aligns cluster operation with renewable availability and carbon intensity signals. For optimization, we deploy the carbon-constrained, performance-constrained, and PPC-driven formulations in the framework. According to each objective, our framework adaptively tunes the carbon emission by adjusting $f(t)$, $n(t)$, and $x(t)$ given workload characteristics.

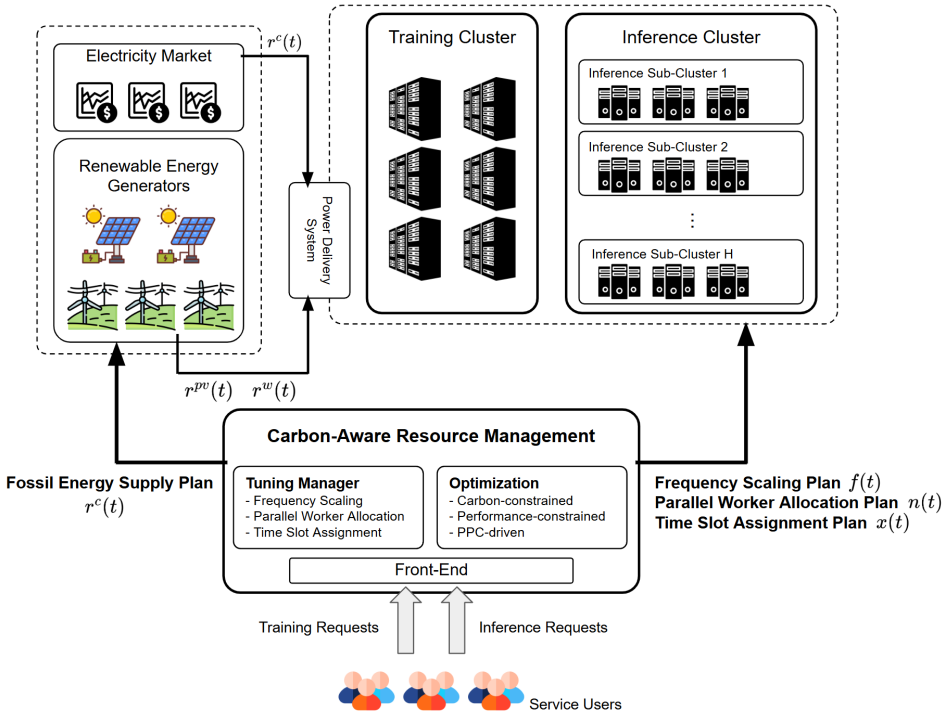


Figure 1. The structure of the proposed Carbon-Aware Resource Management (CA-RM) framework.

3. System Model

In this section, we present the system model that captures job processing performance, cluster-level energy consumption, and carbon emissions associated with both renewable and non-renewable energy sources. We denote by $f^{Tr}(t)$ and $f_i^{Inf}(t)$ the GPU core frequency of the GPU training cluster, and the i-th sub-inference cluster, which corresponds to service level i at time slot t, respectively. In addition, we denote δ as the duration of a time slot.

For simplicity, our framework focuses on tuning only the GPU core frequency while excluding the GPU memory frequency from the decision variables. This design is motivated by the observation that the dynamic power consumption of GPU core processing units

dominates total GPU power, while the memory subsystem contributes a relatively smaller fraction, depending on workload characteristics [20]. In addition, the power–performance behavior induced by GPU memory frequency scaling is qualitatively similar to that of GPU core frequency scaling: increasing the memory frequency leads to higher power consumption and improved performance, analogous to core frequency scaling. Therefore, explicitly including GPU memory frequency as a decision variable provides limited additional optimization benefits while significantly increasing the complexity of the overall optimization problem. To reduce this complexity, we fix the GPU memory frequency at its maximum value and optimize only the GPU core frequency throughout this study.

3.1. Performance Model for Inference Tasks

The hardware specifications and the GPU core frequency values directly influence the service rate for continuously arriving inference requests. Thus, we are able to dynamically tune the service latency of the inference tasks by using the GPU core frequency scaling. Following the models in [21,22], the service latency of the arbitrary inference task in the i-th sub-inference cluster at time slot t, $L_i(t)$ is defined as follows:

$$l_i(t) = \frac{P_{Q=1}}{\rho_i \cdot m_i \cdot f_i^{\mathrm{Inf}}(t) - \lambda_i(t)} + \frac{1}{\rho_i \cdot f_i^{\mathrm{Inf}}(t)} + e_i(t), \quad \forall i, t, \tag{1}$$

Here, $\lambda_i(t)$ and $e_i(t)$ represent the arrival rate of the inference requests to the i-th sub-inference cluster and the corresponding network transmission delay at time slot t, respectively. We assume that all workers remain busy, and thus the queue-related probability term P_Q is always 1. m_i denotes the number of available workers in the i-th sub-inference cluster. In this paper, we do not dynamically adjust m_i; instead, these values are predetermined by the cluster operator. Note that in order to reduce the energy consumption of idle workers, we are able to implement the power-proportional data center through changing m_i. Interested readers may refer to the concept of the dynamic right sizing (DRS) method in [23]. The parameter ρ_i denotes a performance coefficient corresponding to the hardware specifications and the model characteristics of inference requests to the i-th sub-inference cluster. The value of ρ_i differs across inference service categories, which are involved with the DNN model, and clusters due to heterogeneity in both software characteristics and GPU hardware configurations.

In order to penalize the degradation of the inference service latency, we present the penalty function F^{Inf}. The functions F^{Inf} and $f_i^{\mathrm{Inf}}(t)$ are defined as follows:

$$F^{\mathrm{Inf}} = \sum_{\forall t} \sum_{\forall i} F_i^{\mathrm{Inf}}(t), \tag{2}$$

$$F_i^{\mathrm{Inf}}(t) := f_i^{\mathrm{Inf}}(l_i(t) - \hat{l}_i) := \omega_i^{\mathrm{Inf}} \cdot \max\{l_i(t) - \hat{l}_i, 0\}, \ \forall i, t. \tag{3}$$

where $\hat{l}_i$ and ω_i^{Inf} denote the bound of the inference service latency and the weighted value to draw the priority of the inference tasks in the i-th sub-inference cluster, respectively. Note that f_i^{Inf} is the monotonic non-increasing function that linearly penalizes the violation of the service latency bound. The function value f_i^{Inf} is zero when $l_i(t) \geq \hat{l}_i$, as shown in the right term of (3).

3.2. Performance Model for Training Jobs

Training jobs are considered batch workloads, where the overall completion time is more critical than response latency. This characteristic contrasts with inference tasks, which operate as transactional workloads [21]. Therefore, the delayed or deferrable scheduling is acceptable for training jobs, allowing the system to select appropriate time slots for executing each job as long as the predefined training deadline is met. In particular, we

aim to focus on determining the number of allocated workers and the time slots to which training jobs are assigned, in order to adaptively adjust the energy consumption according to the amount of renewable energy generated.

Let τ_j denote the number of completed epochs of the j-th training job within a time slot of fixed duration δ. Following [24,25], we adopt a statistical approach to model the relationship between GPU frequency and per-epoch processing performance as

$$\tau_j(t) = \frac{n_j(t) \times \delta}{\frac{\alpha_{j,0}^{\mathrm{Tr}}}{f^{\mathrm{Tr}}} + \alpha_{j,1}^{\mathrm{Tr}}}, \quad \forall j. \tag{4}$$

where $n_j(t)$ represents the number of parallel GPU workers assigned to the j-th training job at time slot t, while $\alpha_{j,0}^{\mathrm{Tr}}$ and $\alpha_{j,1}^{\mathrm{Tr}}$ are scaling factors specific to the j-th training job's characteristics. These scaling factors can be obtained through offline profiling. Note that f^{Tr} is always fixed as the maximum value, and we do not conduct frequency scaling for the training cluster in contrast to the case of the inference clusters.

Each training job typically requires hundreds of epochs to be trained to achieve a reasonable level of inference accuracy. Let M_j and N denote the number of epochs to be necessarily trained for the j-th training job and the number of available workers in the GPU training cluster, respectively. We set $x_j(t) \in \{0, 1\}$ as the decision variable, which represents whether the time slot t is assigned to j-th training job or not. Then the job completion time of the j-th training job, h_j, is defined as follows:

$$h_j = \mathrm{argmax}_t \{t | x_j(t) = 1, \ \forall t\}, \ \forall j. \tag{5}$$

This value must satisfy

$$h_j \geq a_j, \ \forall j, \tag{6}$$

$$\sum_{\forall t} \tau_j(t) \cdot x_j(t) \geq M_j, \ \forall j, \tag{7}$$

$$\sum_{\forall j} n_j(t) \cdot x_j(t) \leq N, \ \forall t. \tag{8}$$

where a_j denotes the arrival time slot of the j-th training job. Constraint (6) presents that the job completion time should be later than the job arrival time. Constraint (7) presents that the time slot assignment plan should cover the number of required epochs to be trained. Constraint (8) presents that the number of the parallelized workers for entire training jobs cannot exceed the total amount of available GPU resources.

Similar to (2), we define the performance penalty function for training jobs as follows:

$$F^{\mathrm{Tr}} = \sum_{\forall j} F_j^{\mathrm{Tr}}, \tag{9}$$

$$F_j^{\mathrm{Tr}} = f_j^{\mathrm{Tr}}(h_j - \hat{h}_j) := \omega_j^{\mathrm{Tr}} \cdot \max\{h_j - \hat{h}_j, 0\}, \ \forall j. \tag{10}$$

where $\hat{h}_j$ and ω_i^{Tr} denote the deadline and the priority for the j-th training job, respectively. f_j^{Tr} is the penalty function for the j-th training job, similar to (2).

3.3. Energy Consumption Model for Inference Cluster

We provide the power consumption and the associated energy consumption for inference clusters based on the GPU utilization. Following the models presented in [26], the power consumption of the i-th sub-inference cluster, $p_i(t)$, is defined as follows:

$$p_i(t) = \alpha_{i,0}^{\text{Inf}} \cdot f_i^{\text{Inf}}(t) \cdot u_i^{\text{Inf}}(t) + \alpha_{i,1}^{\text{Inf}} \cdot f_i^{\text{Inf}}(t) + \alpha_{i,2}^{\text{Inf}} \cdot u_i^{\text{Inf}}(t) + \alpha_{i,3}^{\text{Inf}}, \ \forall i, t \tag{11}$$

$$= \alpha_{i,0}^{\text{Inf}} \cdot \lambda_i^{\text{Inf}}(t) + \alpha_{i,1}^{\text{Inf}} \cdot f_i^{\text{Inf}}(t) + \alpha_{i,2}^{\text{Inf}} \frac{\lambda_i^{\text{Inf}}(t)}{f_i^{\text{Inf}}(t)} + \alpha_{i,3}^{\text{Inf}}, \ \forall i, t. \tag{12}$$

where $\alpha_{i,0}^{\text{Inf}}, \alpha_{i,1}^{\text{Inf}}, \alpha_{i,2}^{\text{Inf}}, \alpha_{i,3}^{\text{Inf}}$ are model coefficients and can be derived by the offline profiling approach similar to (4). Here, $u_i^{\text{Inf}}(t) = \frac{\lambda_i^{\text{Inf}}(t)}{f_i^{\text{Inf}}(t)}$. In this model, we assume that all the workers in the sub-inference cluster are available and staying in the power-on state.

Based on (12), we present the energy consumption model of the i-th sub-inference cluster as follows:

$$E^{\text{Inf}} = \sum_{\forall i} E_i^{\text{Inf}}, \tag{13}$$

$$E_i^{\text{Inf}} = \sum_{\forall t} E_i^{\text{Inf}}(t), \ \forall i, \tag{14}$$

$$E_i^{\text{Inf}}(t) = \delta \times p_i(t), \ \forall i, \tag{15}$$

where $E_i^{\text{Inf}}(t)$, E_i^{Inf} and E^{Inf} denote the energy consumption of the i-th sub-inference cluster at time slot t, the sum of the energy consumption of the i-th sub-inference cluster for all time slots, and the whole energy consumption of the entire inference clusters, respectively.

3.4. Energy Consumption Model for Training Cluster

To model the energy consumption of the GPU training cluster, we adopt the statistical modeling approach. Based on [25], we present the energy consumption for training jobs as follows:

$$E^{\text{Tr}} = \sum_{\forall j} E_j^{\text{Tr}}, \tag{16}$$

$$E_j^{\text{Tr}} = \sum_{\forall t} E_j^{\text{Tr}}(t), \ \forall j, \tag{17}$$

$$E_j^{\text{Tr}}(t) = n_j(t) \times x_j(t) \times \delta \times (\beta_{j,0}^{\text{Tr}} \cdot f^{\text{Tr}} + \beta_{j,1}^{\text{Tr}}), \ \forall j, t. \tag{18}$$

where $\beta_{j,0}^{\text{Tr}}$ and $\beta_{j,1}^{\text{Tr}}$ are power model coefficients corresponding to the j-th training job. $E_j^{\text{Tr}}(t)$, E_j^{Tr} and E^{Tr} denote the energy consumption of the j-th training job at time slot t, the sum of the energy consumption of the GPU training cluster for all time slots, and the whole energy consumption of all training jobs, respectively. Note that if we do not assign a certain time slot to the training job (i.e., $x_j(t) = 0$), then the involved energy consumption might be zero, as shown in (18). Obviously, we can shift the energy curves of the GPU training cluster by tuning the decision variables $n_j(t)$ and $x_j(t)$.

3.5. Carbon Emission Model

The amount of carbon emission depends on both the amount of energy consumption and what source the energy is generated from. The carbon emission factor is the metric that represents the carbon efficiency of the energy source. Fossil energy such as coal and natural gas has a higher carbon emission factor than renewable energy such as wind and solar, as shown in Table 1. Even within the same renewable energy category, this value might differ according to the detailed sources. Let θ^{c}, θ^{pv} and θ^{w} denote the carbon emission factors for coal-based, solar photovoltaics-based, and wind farm-based energy generation, respectively. Then the total carbon emission $c(t)$ at time slot t can be calculated as follows:

$$c(t) = \sum_{\forall i} c_i^{\text{Inf}}(t) + c^{\text{Tr}}(t), \ \forall t, \tag{19}$$

$$c_i^{\mathrm{Inf}}(t) = \theta^{\mathrm{pv}} \cdot ir_i^{\mathrm{pv}}(t) + \theta^{\mathrm{w}} \cdot ir_i^{\mathrm{w}}(t) + \theta^{\mathrm{c}} \cdot ir_i^{c}(t),\ \forall i, t, \tag{20}$$

$$c^{\mathrm{Tr}}(t) = \theta^{\mathrm{pv}} \cdot tr^{\mathrm{pv}}(t) + \theta^{\mathrm{w}} \cdot tr^{\mathrm{w}}(t) + \theta^{\mathrm{c}} \cdot tr^{c}(t),\ \forall t, \tag{21}$$

$$\sum_{\forall i} ir_i^{\mathrm{pv}}(t) + tr^{\mathrm{pv}}(t) \leq r^{\mathrm{pv}}(t),\ \forall t \tag{22}$$

$$\sum_{\forall i} ir_i^{\mathrm{w}}(t) + tr^{\mathrm{w}}(t) \leq r^{\mathrm{w}}(t),\ \forall t \tag{23}$$

$$\sum_{\forall i} ir_i^{\mathrm{c}}(t) + tr^{\mathrm{c}}(t) \leq r^{\mathrm{c}}(t),\ \forall t \tag{24}$$

$$\sum_{\forall i} E_i^{\mathrm{Inf}}(t) + \sum_{\forall j} E_j^{\mathrm{Tr}}(t) \leq r^{\mathrm{pv}}(t) + r^{\mathrm{w}}(t) + r^{c}(t),\ \forall t. \tag{25}$$

where $r^{\mathrm{pv}}(t)$, $r^{\mathrm{w}}(t)$ and $r^{c}(t)$ represent the amount of generated energy at solar photovoltaics, wind farm, and coal-based generators at time slots t. $ir_i^{\mathrm{pv}}(t)$, $ir_i^{\mathrm{w}}(t)$, and $ir_i^{\mathrm{c}}(t)$ which are solar photovoltaics energy, wind energy, and coal energy delivered to the i-th inference cluster. Similarly, $tr^{\mathrm{pv}}(t)$, $tr^{\mathrm{w}}(t)$, and $tr^{\mathrm{c}}(t)$ are delivered to the training cluster. Note that the sum of each delivered energy cannot exceed the amount of generated energy, as presented in (22)–(24). Constraint (25) presents that the whole energy consumption by both the GPU training cluster and inference clusters should be covered by the total energy generation at every time slot. If the energy amount by the renewable generators is not enough to cover the energy usage, then we should increase $r^{c}(t)$ accordingly.

In this paper, corresponding to the carbon emission, we propose a new metric called performance-per-carbon (PPC) that represents the efficiency of the processing capacity for DNN workloads over the generated carbon. We define the PPC for inference tasks and training jobs as follows, respectively:

$$\overline{\mathrm{PPC}^{\mathrm{Inf}}} = \mathbb{E}\{\sum_{\forall i,t} \mathrm{PPC}_i^{\mathrm{Inf}}(t)\}, \tag{26}$$

$$\mathrm{PPC}_i^{\mathrm{Inf}}(t) = \frac{l_i(t)^{-1}}{c_i^{\mathrm{Inf}}(t)},\ \forall i, t, \tag{27}$$

$$\overline{\mathrm{PPC}^{\mathrm{Tr}}} = \mathbb{E}\{\sum_{\forall t} \mathrm{PPC}^{\mathrm{Tr}}(t)\}, \tag{28}$$

$$\mathrm{PPC}^{\mathrm{Tr}}(t) = \frac{\sum_{\forall j} x_j(t) \times \tau_j(t)}{c^{\mathrm{Tr}}(t)},\ \forall t. \tag{29}$$

Here, $l_i(t)^{-1}$ denotes the inverse of the service latency for tasks executed in the i-th sub-inference cluster at time slot t, and thus corresponds to the throughput of inference processing. Consequently, $\mathrm{PPC}_i^{\mathrm{Inf}}(t)$ and $\mathrm{PPC}^{\mathrm{Tr}}(t)$ represent the number of inference tasks processed on the i-th sub-inference cluster and the number of training epochs completed on the training cluster per carbon emission at time slot t, respectively. $\overline{\mathrm{PPC}^{\mathrm{Inf}}}$ and $\overline{\mathrm{PPC}^{\mathrm{Tr}}}$ denote the average PPC values over whole sub-inference clusters and the training cluster, respectively. Note that the whole energy consumption by both the inference clusters and training cluster cannot exceed the limited carbon budget. Let B denote the carbon budget assigned to the clusters. Then, we define the carbon emission constraints as follows:

$$\sum_{\forall t} c(t) \leq B. \tag{30}$$

Since achieving higher performance typically requires greater energy usage, the carbon budget can be interpreted as the available capacity for assigning computational performance. Note that even for the same amount of energy consumption, the amount of actual carbon emission can be different depending on the energy usage ratio of renewable energy generation. Therefore, under the fixed carbon budget, it is also essential to account for the carbon emission factor in addition to energy consumption. A lower carbon emission factor is more favorable, so it is desirable to schedule and execute as many tasks as possible during periods when renewable energy generation is high and the carbon emission factor is low.

4. Problem Formulation

The goal of our formulation is to minimize carbon emission by jointly optimizing (i) the number of parallel workers and time slot allocation for the training cluster, (ii) the GPU core frequency settings for the inference clusters, and (iii) the amount of energy procured from fossil-based generators, while ensuring acceptable performance for both training jobs and inference tasks. To this end, we present three classes of optimization formulations, the carbon-constrained problem, the performance-constrained problem, and the PPC-driven problem, as detailed in the following subsections.

4.1. Carbon-Constrained Optimization

In this formulation, we aim to simultaneously maximize the performance of both inference tasks and training jobs while ensuring that the total carbon budget is not violated. The problem is formulated as follows:

Problem 1. *Carbon-constrained optimization problem*

$$\min_{n,x,f,ir,tr,r^c} \; F^{Inf} + F^{Tr}, \tag{31}$$

$$s.t. \;\; h_j \geq a_j, \; \forall k, \tag{32}$$

$$\sum_{\forall t} \tau_j(t) \cdot x_j(t) \geq M_j, \; \forall j, \tag{33}$$

$$\sum_{\forall j} n_j(t) \cdot x_j(t) \leq N, \; \forall t, \tag{34}$$

$$\underline{f}_i^{Inf} \leq f_i^{Inf} \leq \overline{f}_i^{Inf}, \; \forall i, \tag{35}$$

$$\sum_{\forall i} ir_i^{pv}(t) + tr^{pv}(t) \leq r^{pv}(t), \; \forall t, \tag{36}$$

$$\sum_{\forall i} ir_i^{w}(t) + tr^{w}(t) \leq r^{w}(t), \; \forall t, \tag{37}$$

$$\sum_{\forall i} ir_i^{c}(t) + tr^{c}(t) \leq r^{c}(t), \; \forall t, \tag{38}$$

$$\sum_{\forall i} E_i^{Inf}(t) + \sum_{\forall j} E_j^{Tr}(t) \leq r^{pv}(t) + r^{w}(t) + r^{c}(t), \; \forall t, \tag{39}$$

$$\sum_{\forall t} c(t) \leq B, \tag{40}$$

$$ir_i(t), tr(t), r^c(t) \geq 0, \; \forall i, t, \tag{41}$$

$$n_j(t) \in \mathcal{N}, \; \forall j, t, \tag{42}$$

$$x_j(t) \in \{0, 1\}, \; \forall j, t. \tag{43}$$

Here, $\underline{f}_i^{\text{Inf}}$ and $\overline{f}_i^{\text{Inf}}$ denote the lower bound and upper bound for frequency scaling of the inference clusters, respectively. The decision variables r^c are automatically minimized depending on r^{pv}, r^{w}, and constraint (40). In Problem 1, we are able to maximize the

processing performance by minimizing the objective terms, F^{Inf} and F^{Tr}, if the carbon budget B is large enough.

4.2. Performance-Constrained Optimization

In this formulation, given predefined performance bounds in terms of service latency and training completion time, our objective is to minimize the carbon emission produced by the clusters. This formulation is applicable when it is not necessary to enforce a strict carbon budget. Even under identical performance levels, the resulting carbon emissions may vary depending on the performance-tuning strategy and job-assignment policy. The problem is formulated as follows:

Problem 2. *Performance-constrained optimization problem*

$$\min_{n,x,f,ir,tr,r^c} \sum_{\forall t} c(t), \tag{44}$$

$$s.t.\ \ h_j \geq a_j,\ \forall k, \tag{45}$$

$$\sum_{\forall t} \tau_j(t) \cdot x_j(t) \geq M_j,\ \forall j, \tag{46}$$

$$\sum_{\forall j} n_j(t) \cdot x_j(t) \leq N,\ \forall t, \tag{47}$$

$$\underline{f}_i^{Inf} \leq f_i^{Inf} \leq \overline{f}_j^{Inf},\ \forall i, \tag{48}$$

$$\sum_{\forall i} ir_i^{pv}(t) + tr^{pv}(t) \leq r^{pv}(t),\ \forall t, \tag{49}$$

$$\sum_{\forall i} ir_i^{w}(t) + tr^{w}(t) \leq r^{w}(t),\ \forall t, \tag{50}$$

$$\sum_{\forall i} ir_i^{c}(t) + tr^{c}(t) \leq r^{c}(t),\ \forall t, \tag{51}$$

$$\sum_{\forall i} E_i^{Inf}(t) + \sum_{\forall j} E_j^{Tr}(t) \leq r^{pv}(t) + r^{w}(t) + r^{c}(t),\ \forall t, \tag{52}$$

$$l_i(t) \leq \hat{l}_i,\ \forall i, t, \tag{53}$$

$$h_j \leq \hat{h}_j,\ \forall j, \tag{54}$$

$$ir_i(t), tr(t), r^c(t) \geq 0,\ \forall i, t, \tag{55}$$

$$n_j(t) \in \mathcal{N},\ \forall j, t, \tag{56}$$

$$x_j(t) \in \{0, 1\},\ \forall j, t. \tag{57}$$

In contrast to Problem 1, in the formulation above, the inference latency and training completion time are considered tight constraints, as presented in (53) and (54). In this formulation, we assume that we are always able to find the feasible solution that enables avoiding violation of the performance constraints.

4.3. PPC-Driven Optimization

In this formulation, our goal is to maximize our novel PPC metric presented above, without consideration for performance bound and carbon budget limitations; thus, we are able to focus only on the carbon efficiency. For the training cluster, we can apply the delayed scheduling of training jobs toward the carbon-efficient time slots (i.e., when the amount of renewable energy generation is high). For the inference clusters, we can apply adaptive frequency scaling of computing workers in real time. To do this, the problem is formulated as follows:

Problem 3. *PPC-driven optimization problem*

$$\min_{n,x,f,ir,tr,r^c} \ \omega^{Inf} \cdot \overline{PPC^{Inf}} + \omega^{Tr} \cdot \overline{PPC^{Tr}}, \tag{58}$$

$$s.t. \ \ h_j \geq a_j, \ \forall k, \tag{59}$$

$$\sum_{\forall t} \tau_j(t) \cdot x_j(t) \geq M_j, \ \forall j, \tag{60}$$

$$\sum_{\forall j} n_j(t) \cdot x_j(t) \leq N, \ \forall t, \tag{61}$$

$$\underline{f}_i^{Inf} \leq f_i^{Inf} \leq \overline{f}_j^{Inf}, \ \forall i, \tag{62}$$

$$\sum_{\forall i} ir_i^{pv}(t) + tr^{pv}(t) \leq r^{pv}(t), \ \forall t, \tag{63}$$

$$\sum_{\forall i} ir_i^{w}(t) + tr^{w}(t) \leq r^{w}(t), \ \forall t, \tag{64}$$

$$\sum_{\forall i} ir_i^{c}(t) + tr^{c}(t) \leq r^{c}(t), \ \forall t, \tag{65}$$

$$\sum_{\forall i} E_i^{Inf}(t) + \sum_{\forall j} E_j^{Tr}(t) \leq r^{pv}(t) + r^{w}(t) + r^{c}(t), \ \forall t, \tag{66}$$

$$ir_i(t), tr(t), r^c(t) \geq 0, \ \forall i, t, \tag{67}$$

$$n_j(t) \in \mathcal{N}, \ \forall j, t, \tag{68}$$

$$x_j(t) \in \{0, 1\}, \ \forall j, t. \tag{69}$$

Here, ω^{Inf} and $\omega^{\mathrm{Tr}} := 1 - \omega^{\mathrm{Inf}}$ denote the processing priority for inference and training, respectively. Note that in contrast to Problems 1 and 2, this problem does not include the terms for bounds like (40), (53), and (54). If the carbon budget is high enough and the performance bound is loose, then this formulation can optimize both the carbon efficiency and the processing performance for DNN workloads, simultaneously.

4.4. DNN-Based Surrogate Optimizer

Note that Problems 1–3 are all nonlinear and non-convex due to terms such as the multiplication of decision variables in (7) and (8), as well as the reciprocal of decision variables in (27). To address this challenge, we adopt a meta-heuristic approach, which enables approximate optimization for non-convex formulations. In particular, we employ a variant of the DNN-based surrogate optimizer presented in our previous work [27]. The DNN-based surrogate optimizer enables deriving the sufficiently approximated optimal solution even for large-scale non-convex problems by simply transforming the original constrained problems to unconstrained ones. For example, we define the unsupervised DNN loss function based on Problem 1 as follows:

$$\Theta_{\mathrm{carbon}}(\mathbf{v}) = F^{\mathrm{Inf}}(\mathbf{s} = DNN(\mathbf{v}; \mathbf{w})) + F^{\mathrm{Tr}}(\mathbf{s} = DNN(\mathbf{v}; \mathbf{w})) + \Phi_{\mathrm{carbon}}(\mathbf{s}). \tag{70}$$

where

$$\Phi_{\text{carbon}}(\mathbf{s}) = \phi_d^{\text{carbon}} \sum_{\forall j} (a_j - h_j)^+ + \phi_M^{\text{carbon}} \sum_{\forall j} (\sum_{\forall t} M_j - \tau_j(t) \cdot x_j(t))^+ \tag{71}$$

$$+ \phi_N^{\text{carbon}} \sum_{\forall t} (\sum_{\forall j} n_j(t) \cdot x_j(t) - N)^+ \tag{72}$$

$$+ \phi_{\underline{f}}^{\text{carbon}} \sum_{\forall i} (\underline{f}_i^{\text{Inf}} - f_i^{\text{Inf}})^+ + \phi_{\overline{f}}^{\text{carbon}} \sum_{\forall i} (f_i^{\text{Inf}} - \overline{f}_i^{\text{Inf}})^+ \tag{73}$$

$$+ \phi_{r^{\text{pv}}}^{\text{carbon}} \sum_{\forall i} (ir_i^{\text{pv}}(t) + tr^{\text{pv}}(t) - r^{\text{pv}}(t))^+ \tag{74}$$

$$+ \phi_{r^{\text{w}}}^{\text{carbon}} \sum_{\forall i} (ir_i^{\text{w}}(t) + tr^{\text{w}}(t) - r^{\text{w}}(t))^+ \tag{75}$$

$$+ \phi_{r_1^{\text{c}}}^{\text{carbon}} \sum_{\forall i} (ir_i^{\text{c}}(t) + tr^{\text{c}}(t) - r^{\text{c}}(t))^+ \tag{76}$$

$$+ \phi_e^{\text{carbon}} \sum_{\forall t} (\sum_{\forall i} E_i^{\text{Inf}}(t) + \sum_{\forall j} E_j^{\text{Tr}}(t) - (r^{\text{pv}}(t) + r^{\text{w}}(t) + r^c(t)))^+ \tag{77}$$

$$+ \phi_b^{\text{carbon}} (\sum_{\forall t} c(t) - B)^+ \tag{78}$$

$$+ \phi_{ir}^{\text{carbon}} \sum_{\forall i,t} (-ir_i(t))^+ \tag{79}$$

$$+ \phi_{tr}^{\text{carbon}} \sum_{\forall t} (-tr(t))^+ \tag{80}$$

$$+ \phi_{r_2^{\text{c}}}^{\text{carbon}} \sum_{\forall t} (-r^c(t))^+, \tag{81}$$

and $\phi_d^{\text{carbon}} - \phi_{r_2^{\text{c}}}^{\text{carbon}}$ are penalty weight values for constraint violation. $\mathbf{v}$, $\mathbf{w}$ and $\mathbf{s}$ denote the input data (i.e., user requests and renewable generation) to the DNN model weight parameters and output data (i.e., decision variables n, x, f, ir, tr, r^c), respectively. Here, $(x)^+ = \max(x, 0)$. The DNN loss functions based on Problems 2 and 3 can be derived in the same way presented above. Interested readers may refer to [27] for further details.

4.5. Proposed Algorithm

Algorithm 1 presents the overall procedures for deriving the optimal decision variables n, x, f, ir, tr, and r^c for each target problem. Given model coefficients and constants, our framework efficiently finds the approximate decision variables corresponding to target formulations.

Algorithm 1: Optimization of the CA-RM Framework

INPUT:

Model coefficients and constants ρ, m, e, a, ω, δ, α^{Tr}, α^{Inf}, β^{Tr}, M, N, B, $\underline{f}$, $\overline{f}$, ϕ;
Performance bound for inference, $\hat{l}$ and for training, $\hat{h}$;
Carbon emission factors θ;

OUTPUT:

Optimal decision variables, n, x, f, ir, tr, r^c.

STEPS:

1. Predict the renewable generation r^{pv} and r^{w}.
2. Select the target formulation among Problems 1–3.
3. Find the approximate optimal solution for the target formulation by using the DNN-based surrogate optimizer [27].
4. Return the optimal solution n^*, x^*, f^*, ir^*, tr^*, r^{c*}.

5. Experiments

5.1. Cluster and Workload Setup

To conduct extensive simulation-based experiments, we empirically collected real processing data for DNN model training and inference from the computing node equipped with the NVIDIA RTX4090 GPU. The detail of the node is shown in Table 2. We deployed four well-known DNN models, vgg16, inceptionv4, resnet152, and stochasticdepth101, and executed both training jobs and inference tasks for each model, with the dataset CIFAR-100 [28]. The details of each DNN model are shown in Table 3. To execute DNN model training jobs and inference tasks, we set the computing node with CUDA 13.0, cuDNN 9.15, and PyTorch 2.9.0+cu130. We measured the training and inference time, as well as the associated energy consumption, under various GPU core clock configurations. To systematically adjust the GPU core clock values, we utilized the NVIDIA Management Library (NVML) [29] and implemented the Python (3.14)-based scripts that automate the frequency scaling process, thereby overriding the default GPU frequency behavior. We conducted GPU frequency scaling over the 5five available values for the RTX4090 device (210 MHz, 810 MHz, 1410 MHz, 2010 MHz, 2760 MHz). After configuring each device, our customized logging modules continuously recorded instantaneous power draw, training time, and inference latency throughout the experiments.

Table 2. Specification of the RTX4090-based computing node.

CPU	Mainboard	RAM	DISK	OS	GPU
i9-13900KF (32 cores)	Z790-P WIFI	DDR5 64 GB	SHPP41-1000 GM 1 TB	Windows10	RTX4090 (GDDR6X 24 GB, 16,384 CUDA cores)

Table 3. DNN model specification.

Info	VGG16 [30]	Inceptionv4 [31]	Resnet152 [32]	Stochasticdepth101 [33]
Architecture	Plain CNN	Inception	Residual CNN	Residual CNN
Depth (Layers)	16	75	152	101
Parameters	∼138 M	∼43 M	∼60 M	∼44 M
Use Cases	Feature Extraction	Image Classification	Image Recognition	Robust CNN Training

Using the empirical testbed results obtained from our RTX4090-based node, we conduct an extensive simulation to compare the proposed CA-RM framework with previous studies. The simulated GPU cluster consists of 100 RTX4090-based nodes (50 nodes in the training cluster and remain 50 nodes in the inference clusters). The time slot duration is set to 15 min, and the total simulation period is 24 h. The generated DNN workload behaviors are grouped into three categories: light workloads (inceptionv4 0.1, resnet152 0.1, stochasticdepth101 0.2, vgg16 0.6), medium workloads (inceptionv4 0.2, resnet152 0.2, stochasticdepth101 0.2, vgg16 0.4), and heavy workloads (inceptionv4 0.4, resnet152 0.4, stochasticdepth101 0.1, vgg16 0.1). The deadlines for training and latency bounds for inference are configured according to the characteristics of each DNN model. For renewable energy modeling, we use the practical data obtained from [34]. In particular, we incorporate solar irradiance, ambient temperature, and wind speed data collected during 24 h on 1 October 2018, from the NREL-M2 site in Boulder, CO, USA.

5.2. Evaluation Metric

To quantitatively evaluate the performance of the proposed CA-RM framework, we define the following metrics:

(1) Training Completion Time and Inference Throughput: The training completion time represents the total time required to complete the assigned number of training epochs. For each deployed DNN model, we obtain the training time using our Python-based parsing scripts. The inference throughput measures the number of inference requests processed per unit time. We use one minute as the evaluation interval. The inverse of inference throughput corresponds to the service latency experienced by inference requests.
(2) Service Level Agreement (SLA) Violation: This metric evaluates the reliability of our CA-RM framework with respect to both training and inference processing performance. Note that when a predefined SLA (e.g., service latency bound and training deadline) must be satisfied, maximizing performance is not always necessary. Instead, it is desirable to maintain performance at the level required to meet the SLA while reducing operational costs such as energy consumption and carbon emission.
(3) Energy Consumption: This metric quantifies the energy consumed during DNN model training and inference processing. Although power consumption is generally inversely related to processing performance, energy usage does not always draw a simple behavior. The configuration that minimizes energy consumption varies depending on node specifications, DNN model architecture, and the characteristics of the input dataset.
(4) Carbon Emission and Efficiency: The metric of carbon emission measures the amount of carbon emitted during DNN workload processing on GPU clusters. The carbon emission is closely tied to energy usage, but even with identical energy consumption, the resulting emissions differ depending on the energy source. Maximizing the utilization of renewable energy is therefore essential to reducing the carbon emission. The carbon efficiency is based on the proposed PPC metrics defined in (27) and (29), and is used to evaluate how effectively the framework minimizes the carbon emission while achieving acceptable processing performance. Higher PPC values indicate greater carbon efficiency. However, note that maximizing PPC does not inherently guarantee adherence to a given carbon budget. To ensure that the carbon budget is not violated, Problem 1 must be applied, potentially at the cost of reduced carbon efficiency.

5.3. Competitors

In order to verify the performance of our CA-RM framework, we compare our proposed approach to two previous studies and two baselines. The first previous method, EnergyCost [21], presents the deep learning job (containing training and inference) allocation considering both the electricity price and the heterogeneity of jobs. This method considers the dynamic electricity price from the grid market and conducts adaptive available time slot allocation accordingly. In addition, it adopts the dynamic right sizing (DRS) approach that turns the idle servers off, in order to avoid the waste of the unnecessary power usage of clusters. This method exploits the mixed-integer nonlinear problem (MINLP) formulation so as to delicately control the trade-off between the processing performance and energy usage cost. Although this approach shows noticeable energy saving while guaranteeing acceptable DNN model processing performance, it cannot explicitly consider the carbon emission associated with the energy usage. It does not conduct budget-based performance throttling like ours, so it cannot avoid unacceptable carbon emission from the cluster. The second previous method, PowerCap [25], tries to optimize the DNN model training performance without the violation of the predefined power budget, through real-time-based GPU core and memory frequency scaling based on economic model predictive control (EMPC). It utilizes recursive least square-based model estimation to capture the proper model coefficients for frequency scaling without time consuming offline profiling processes.

In particular, this method adopts a distributed optimization structure so as to enable timely control even for hundreds of GPU servers. However, this method only considers the model training and not the inference tasks that are not deferrable compared to model training jobs that are allowed to be delayed within the deadline. Additionally, this method only focuses on power consumption, and not energy consumption and involved carbon emission that are formulated in nonlinear forms cannot be easily solved.

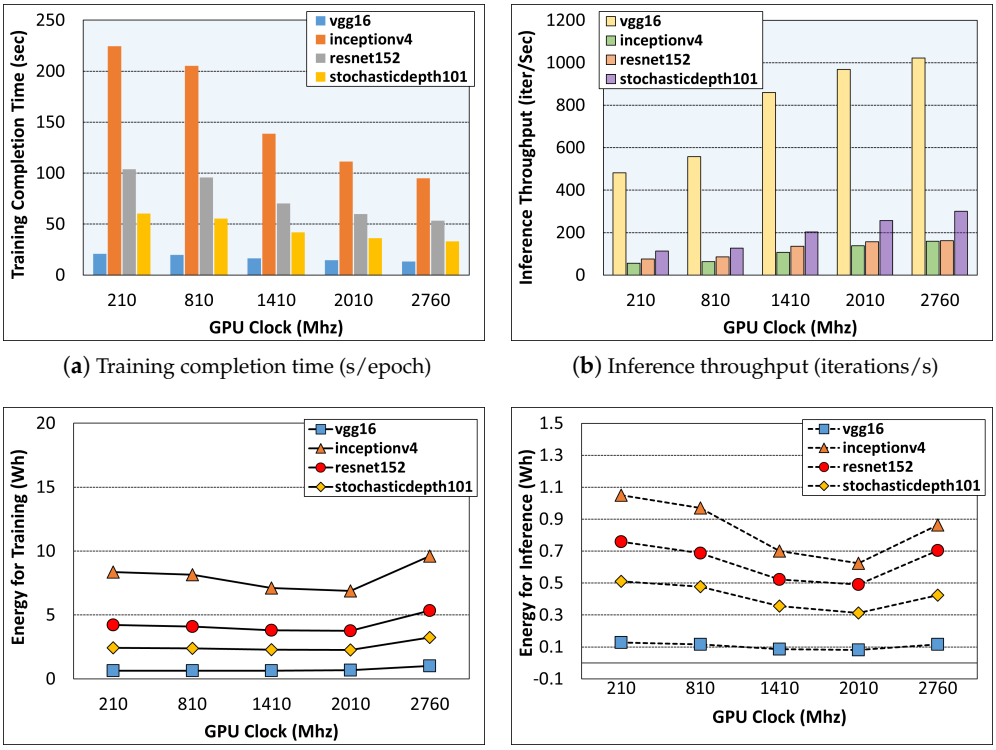
As baseline approaches, we conduct the analysis under the baseline-MIN and baseline-MAX. In the baseline-MIN, we fix the GPU core clocks to the minimum values for all the DNN workloads. Regardless of the deadline and latency bound, GPU core clocks are fixed during the time slots. This method enables the minimization of power consumption derived by GPU devices, but cannot ensure optimized energy consumption. When the DNN model training time significantly increases, the energy consumption may be undesirably increased. In the baseline-MAX, we fix the GPU core clocks to the maximum values for all the DNN workloads. In contrast to the baseline-MIN, this method maximizes the power consumption of GPU devices and the processing capacity as well. It enables maximizing the processing performance, but does not ensure optimized energy consumption and involved carbon emission due to the unacceptably high power consumption. Note that the time slot allocation decision-making process presented in Algorithm 1 is not applied to the baseline-MIN and baseline-MAX approaches. Instead, in these baseline approaches, user requests are allocated sequentially upon arrival to available workers and the nearest available empty time slots.

5.4. Evaluation Results

Figure 2a,b depict the DNN model training completion time and the inference throughput, corresponding to the change of GPU core frequency values on an RTX4090-based node. Obviously, the processing performance improves as the GPU core clock increases since the GPU processing capacity also increases. It is worth noting that the performance variation due to GPU frequency scaling differs significantly depending on the computational characteristics of DNN models. For instance, as shown in Figure 2a, the inceptionv4 model takes over 220 s to process a single epoch at the lowest GPU core clock (210 MHz), whereas the time drops to under 100 s at the highest clock (2760 MHz), resulting in a performance gain of more than 50%. In contrast, the vgg16 model shows a smaller improvement—from 20 s to 13 s—across the same clock range, reflecting only about a 35% increase in performance. This discrepancy across models is primarily determined by the proportion of GPU core operations in the overall training computation, that is, the compute intensity of the model. Compared to inceptionv4 (about 41 M parameters), vgg16 (about 34 M parameters) is a smaller model, and a greater portion of its execution time is attributed to CPU operations and data transfer between the host and GPU memory, rather than massively parallel computations on the GPU. As a result, increasing the GPU core clock has a more limited impact on the overall training performance. The inference throughput results in Figure 2b also reflect this pattern.

We can see the energy consumption for both training and inference processing over GPU core clock configurations in Figure 2c,d. In contrast to performance results, the energy usage curves are shown in somewhat differently complex manners. The energy consumption curve for training shown in Figure 2c does not exhibit a monotonic trend with respect to the GPU core clock. Notably, the minimum energy usage occurs at a core clock of 2010 MHz, not at the minimum one (210 MHz). Although both power consumption and processing performance decrease monotonically as the core clock is reduced, the resulting energy consumption does not follow the same monotonic pattern. This discrepancy arises because energy consumption is defined as the product of power and processing time, and

the rate at which power decreases and the rate at which processing time increases under lower core clock settings may differ. When the degradation in performance (i.e., the increase in processing time) outweighs the power reduction, the total energy consumption can in fact increase. Consequently, merely lowering the core clock does not guarantee improved energy efficiency; delicate profiling is required to find the accurate optimal frequency values. The energy consumption curve for inference in Figure 2d exhibits a trend similar to that observed in Figure 2c. Across all evaluated DNN models, the optimal GPU core clock that yields the lowest energy consumption is 2010 MHz. In contrast, both the lowest clock setting (210 MHz) and the highest setting (2760 MHz) result in noticeably inferior energy efficiency compared with other intermediate clock configurations.



(**a**) Training completion time (s/epoch)

(**b**) Inference throughput (iterations/s)

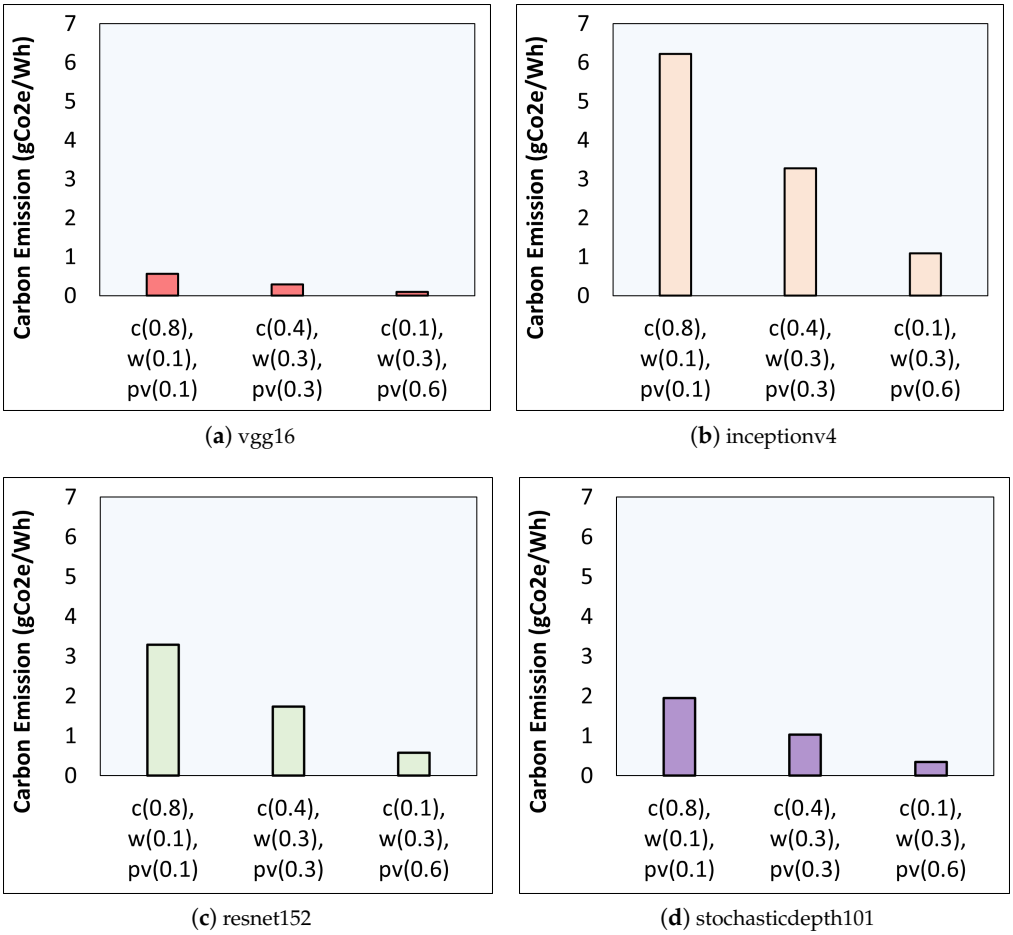
(**c**) Energy consumption for training (Wh, GPU power × training completion time)

(**d**) Energy consumption for inference (Wh, GPU power × inference latency)

Figure 2. Performance and energy consumption of RTX4090-based node to process DNN models: vgg16, inceptionv4, resnet152, and stochasticdepth101.

Figure 3a–d present the average carbon emissions generated during the training of each DNN model under various energy source configurations. We vary the proportion of coal, wind, and solar photovoltaics energy contributing to the total energy usage. For instance, the configuration (c(0.8), w(0.1), pv(0.1)) indicates that 80% of the required energy is supplied by coal, 10% by wind, and 10% by solar photovoltaics. A higher proportion of wind and solar photovoltaics corresponds to a greater share of renewable energy generation. Across all models, we observe a consistent trend: carbon emissions decrease as the renewable energy contribution increases. For vgg16, carbon emissions are reduced from 0.08 gCO_2e/Wh under the configuration (c(0.8), w(0.1), pv(0.1)) to 0.015 gCO_2e/Wh under the configuration (c(0.1), w(0.3), pv(0.6)), an approximately 80% reduction. Similar levels of reduction are observed for the other models, including inceptionv4, resnet152, and stochasticdepth-101. These findings suggest that substantial improvements in carbon efficiency can be achieved solely by shifting the timing of training jobs or inference tasks to periods with higher renewable energy availability, without requiring any modifications to

GPU hardware efficiency or DNN model architectures. Figure 4a–d depict the average carbon emissions associated with inference tasks under the same energy source configurations, showing similar trends to those in Figure 3.



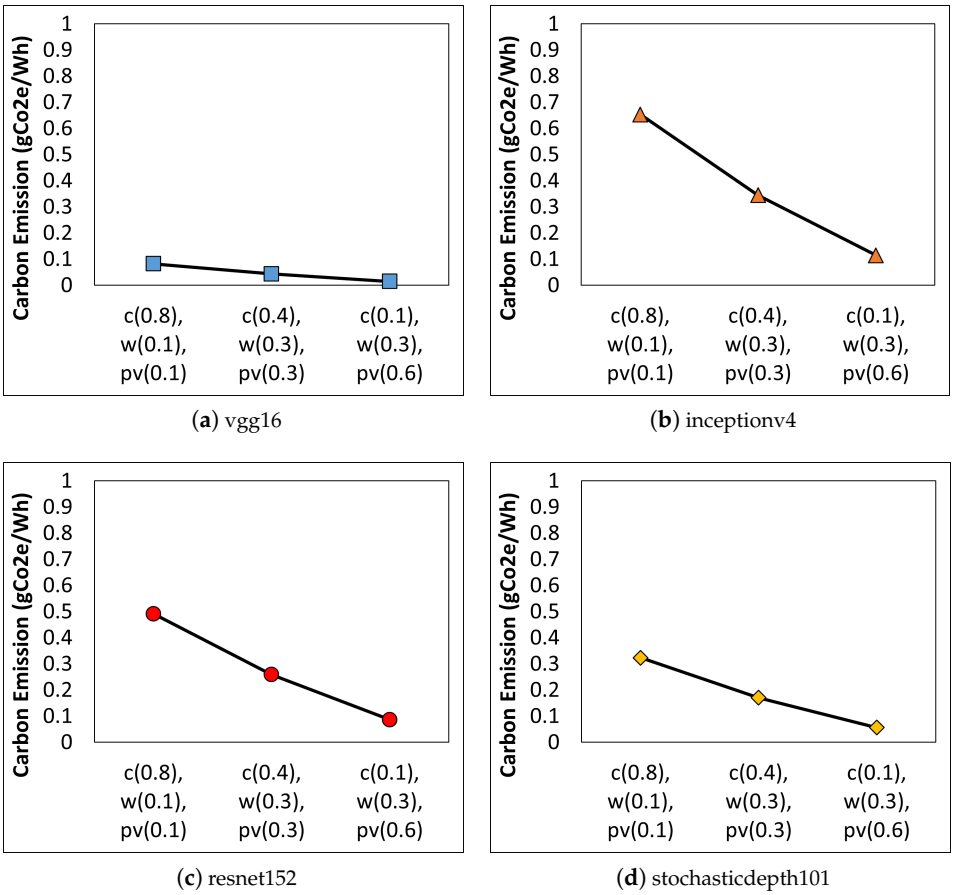
(**a**) vgg16 (**b**) inceptionv4

(**c**) resnet152 (**d**) stochasticdepth101

Figure 3. Carbon emission of RTX4090-based node for DNN model Training.

We compare our proposed CA-RM framework to previous methods presented earlier in view of carbon emission and efficiency (PPC), as shown in Figure 5. Figure 5a shows the carbon emission of two baseline approaches (MAX and MIN), two existing methods, EnergyCost [21] and PowerCap [25], and our proposed approaches including carbon-constrained, performance-constrained, and PPC-driven optimization. The unit of relative carbon emissions in Figure 5a is normalized with respect to the carbon emissions produced by the baseline-MAX approach given heavy workloads. Specifically, a unit value of 1 corresponds to 154,688 gCO_2e/Wh. For light workloads, our proposed carbon-constrained approach achieves the best performance among all methods, as it focuses exclusively on adhering to the carbon budget. The performance-constrained and PPC-driven approaches exhibit similar outcomes because the workloads are not computationally intensive, and the resulting carbon emissions remain relatively low. Under such conditions, these methods can maximize the processing performance without incurring significant carbon emission. The EnergyCost method also performs favorably compared with our CA-RM framework, as it reduces energy cost by prioritizing the use of available renewable energy. This strategy indirectly leads to lower carbon emissions. In contrast, the PowerCap method yields carbon inefficiency because it focuses on limiting instantaneous power consumption rather than total energy consumption, which makes it ineffective at reducing cumulative carbon emissions. The two baseline methods, MAX and MIN, per-

form unfavorably from a carbon efficiency perspective. The baseline-MIN approach, which applies the minimum GPU core clock, significantly increases training time and inference latency, thereby raising total energy consumption. The baseline-MAX approach uses the maximum GPU core clock setting, which leads to substantially higher power consumption while offering only marginal improvements in DNN processing performance.

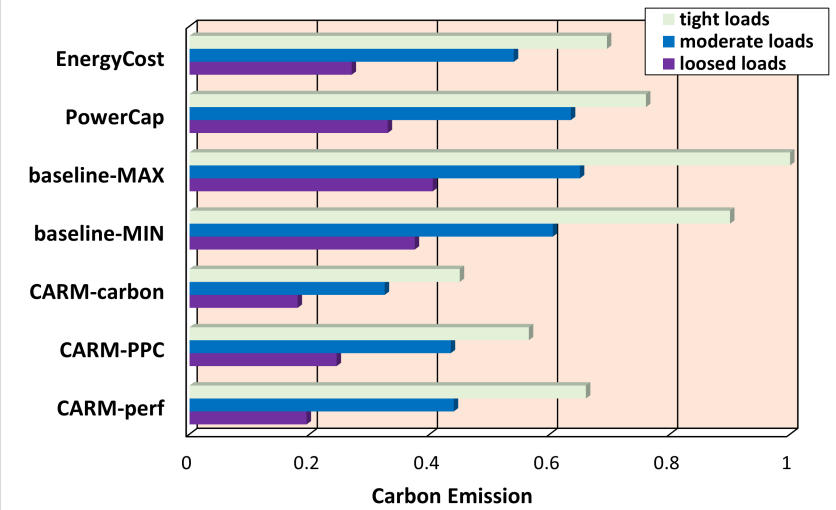


(**a**) vgg16 (**b**) inceptionv4

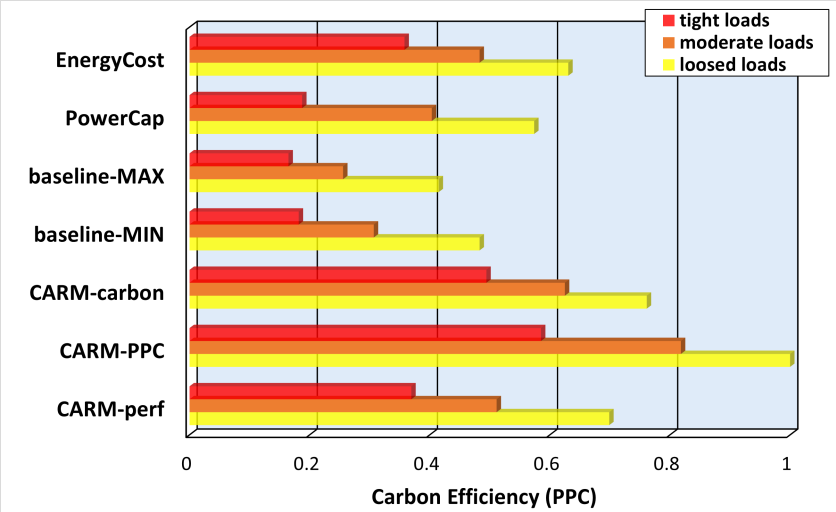
(**c**) resnet152 (**d**) stochasticdepth101

Figure 4. Carbon emission of RTX4090-based node for DNN model inference.

For medium workloads, the experimental results exhibit carbon emission patterns that are consistent with those observed under light workloads. However, compared with the light workload case, both the PPC-constrained and performance-constrained approaches generate substantially higher carbon emissions. This increase arises because the larger volume of tasks leads to a higher absolute amount of energy consumption. The results produced by the EnergyCost method under medium workloads remain similar to those for light workloads. Since the EnergyCost method consistently aims to minimize energy cost by maximizing the utilization of renewable energy, its carbon emissions also tend to decrease correspondingly. Under medium workloads, the PowerCap method applies stricter power capping than in the light workload case due to the heavier processing demand. While this stricter capping reduces instantaneous power consumption, it also increases inference latency and training completion time, making the associated carbon emission non-negligible.



(**a**) Relative carbon emission



(**b**) Relative carbon efficiency

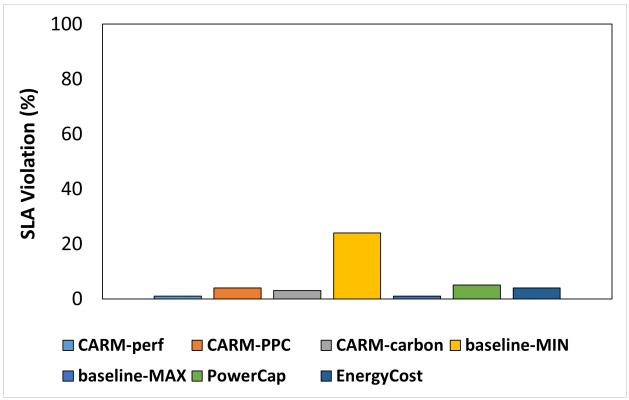
Figure 5. Comparison of the proposed CA-RM framework to EnergyCost [21], PowerCap [25], and baselines in views of relative carbon emission and carbon efficiency (PPC).

For heavy workloads, the carbon-constrained approach continues to achieve the lowest carbon emissions because it strictly adheres to the predefined carbon budget, even at the expense of DNN processing performance. Although the performance-constrained approach also attempts to reduce carbon emissions using (44), it results in higher emission than the carbon-constrained method since maintaining the performance bound is treated as the highest priority during resource allocation. The PPC-driven approach also yields the high emission because it focuses solely on optimizing the PPC metric defined in (58) and does not explicitly account for the absolute carbon emission level or the available carbon budget. Note that under heavy workload conditions, a large number of user requests must be processed within a short time window, which inevitably requires increasing the processing capacity of computing nodes. Higher energy consumption and increased carbon emissions are unavoidable in such cases. Unlike the carbon-constrained approach whose primary objective is to strictly limit absolute carbon emissions, the PPC-driven approach is designed to maximize performance per carbon emission rather than minimizing total carbon output. From this perspective, an increase in carbon emissions under heavy workloads when using the PPC-driven approach is an expected outcome. Both the EnergyCost and PowerCap

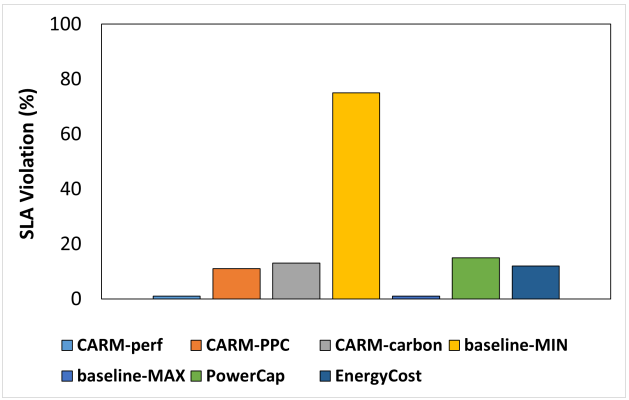
methods lead to relatively high carbon emissions under heavy workloads, as they operate on large volumes of DNN jobs and inference tasks that inherently require substantial energy consumption. Among all approaches, the baseline-MAX method performs the worst in terms of carbon emission because it consistently triggers maximum power consumption for all training jobs and inference tasks. The baseline-MIN method performs better than baseline-MAX, as operating at the minimum GPU core clock reduces power consumption and thus lowers total energy usage. Note that, across all workloads, the proposed approaches achieve an average carbon emission of approximately 60,000 gCO_2e/Wh. This corresponds to an improvement of 35% compared to all other competing methods, and an improvement of 28% relative to the PowerCap and EnergyCost approaches. Figure 5b compares the carbon efficiency quantified using the PPC values in (27) and (29), across our proposed approaches and the other competitors. Obviously, the PPC-driven approach consistently achieves the highest carbon efficiency for all workload cases because it focuses only on optimizing the PPC value without explicitly considering performance constraints or carbon budget limits. From the perspective of carbon efficiency, the carbon-constrained approach outperforms the performance-constrained approach. This is because the carbon-constrained approach directly limits the absolute carbon emissions through a predefined carbon budget, whereas the performance-constrained approach prioritizes meeting the required DNN processing performance bounds, even at the cost of incurring higher carbon emissions. Both the EnergyCost and PowerCap methods exhibit relatively high carbon emissions, as neither explicitly accounts for carbon efficiency in their optimization objectives. The baseline-MAX and baseline-MIN methods show the lowest carbon efficiency regardless of the workload characteristics, since they rely on fixed-frequency settings and therefore lack any adaptive carbon-aware tuning capability.

Figure 6a–c present the average SLA violation of each approach for both training and inference processing across the three workload characteristics. As shown in Figure 6a, all approaches except baseline-MIN satisfy the performance bounds under the light workload. The performance-constrained approach results in near-zero SLA violations because it explicitly enforces both the training deadline and the inference latency bound. The carbon-constrained approach also effectively prevents SLA violations, as the assigned carbon budget is sufficiently large to accommodate the DNN workloads. The PPC-driven approach exhibits a small amount of SLA violation due to the absence of an explicit performance constraint, focusing solely on maximizing the PPC metric. The baseline-MAX method, as expected, incurs no SLA violations because it consistently maintains the maximum GPU core clock configuration across all combinations of renewable energy availability and model types. In contrast, baseline-MIN produces the worst SLA performance, as fixing the GPU to the minimum core clock leads to substantial performance degradation across all cases. Figure 6b,c reveal results consistent with those in Figure 6a. Under medium and heavy workloads, our proposed approaches, except for the performance-constrained method, show SLA violation levels comparable to those of the EnergyCost and PowerCap methods (approximately 15% and 30%, respectively). This stems from the fact that the carbon-constrained and PPC-driven approaches prioritize reducing carbon emissions or improving carbon efficiency, even at the cost of certain performance degradation. As shown in Equations (31)–(43), the carbon-constrained formulation does not explicitly impose constraints on DL job performance requirements (i.e., the training deadline $\hat{h}$ and inference latency bound $\hat{l}$). Instead, this formulation explicitly enforces compliance with a predefined carbon budget, as expressed in Equation (40), and subsequently seeks to maximize processing performance (i.e., F^{Inf} and F^{Tr}) within the feasible carbon budget range. In contrast, as shown in Equations (44)–(57), the performance-constrained formulation explicitly incorporates the DL job performance requirements as hard constraints, and

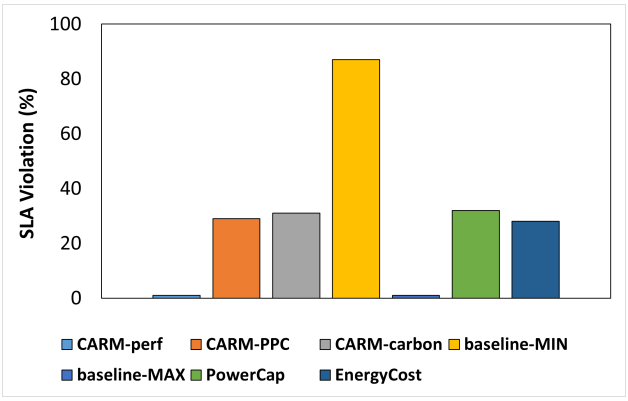
then minimizes carbon emissions $c(t)$ within the feasible performance range. Under light workload conditions, the performance requirements are relatively modest, allowing the carbon-constrained approach to achieve low SLA violation even under a limited carbon budget. However, under medium and heavy workloads, satisfying stricter performance requirements inevitably leads to increased energy consumption and carbon emissions. As a result, the approaches that do not explicitly enforce SLA constraints, except for the performance-constrained formulation, can incur substantial SLA violations. This issue can be mitigated by increasing the carbon budget B in the carbon-constrained formulation. In the PPC-driven formulation, we can redesign the PPC metric in Equations (27) and (29) by adding the exponent coefficient to the $c(t)$ of the denominator (e.g., $\{c(t)\}^2$) so as to differentiate each weight value to performance and carbon emission. We will address this topic in future work. Nevertheless, it is important to note that our proposed approaches do not incur greater performance degradation than the existing methods. Overall, the results in Figure 6 demonstrate that our approaches within the CA-RM framework are available for GPU cluster management, effectively balancing carbon emission reduction with acceptable DNN processing performance.



(**a**) Light workloads



(**b**) Medium workloads



(**c**) Heavy workloads

Figure 6. Service-level agreement (SLA) violation given (light, medium, and heavy) workload characteristics.

6. Conclusions

In this paper, we present a novel Carbon-Aware Resource Management (CA-RM) framework maximizing the utilization of renewable energy, in order to achieve carbon-efficient GPU cluster operation with DNN model processing. We introduce a new metric called performance-per-carbon (PPC) so as to quantitatively evaluate the carbon efficiency. We utilize the GPU core frequency scaling and allocation planning of computing nodes as

energy-shifting methodologies adaptive to dynamically renewable generation behaviors. To achieve the delicate tuning of both carbon emission and DNN workload processing performance simultaneously, we design the carbon-constrained, performance-constrained, and PPC-driven optimization formulations based on sophisticated mathematical modeling. To validate the availability of the CA-RM framework, we retrieved the profiling data from the NVIDIA RTX4090-based node with well-known DNN models, vgg16, inceptionv4, resnet152, and stochasticdepth101, and conducted extensive simulation experiment. Through the experimental results, we demonstrate that our proposed CA-RM framework outperforms recent existing methods for GPU cluster management. The CA-RM framework achieves carbon reduction of more than 35% on average, compared to its competitors while guaranteeing acceptable DNN workload processing performance. We verify that our work is proper for recent modern sustainable AI data centers.

We note that, despite the experimental results demonstrating that the CA-RM framework achieves carbon-efficient deep learning job allocation, the scope of this work is limited in terms of model generality. Specifically, the proposed CA-RM framework is designed for conventional deep learning models, such as DNNs and CNNs, and is not directly applicable to LLMs, including Transformer-based neural networks. Transformer-based models employ deeply stacked and recursive layer structures that differ fundamentally from those of traditional DNNs and CNNs. As a result, accurately capturing their performance and carbon emission characteristics would require redesigning the underlying system models used in our framework. Extending the proposed CA-RM framework to support state-of-the-art LLM workloads is an important direction for future work, and we plan to investigate such extensions in subsequent studies. In addition, we focus exclusively on validating the carbon efficiency of the CA-RM framework implemented in-house, and we do not consider its integration with general-purpose cluster management frameworks. As for future research, we plan to extend the CA-RM framework to be deployed as a plug-in for widely adopted cluster management frameworks, such as Kubernetes and Slurm, and to experimentally evaluate its performance and practicality in real-world cluster environments.

Author Contributions: Conceptualization, D.-K.K. and Y.-H.M.; Methodology, D.-K.K. and Y.-H.M.; Software, D.-K.K.; Validation, D.-K.K.; Formal analysis, D.-K.K.; Investigation, D.-K.K.; Resources, D.-K.K.; Writing—original draft, D.-K.K.; Writing—review & editing, D.-K.K. and Y.-H.M.; Supervision, Y.-H.M.; Project administration, Y.-H.M.; Funding acquisition, Y.-H.M. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Sungshin Women's University Research Grant of 2025.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available in the article.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

DL	Deep Learning
CA-RM	Carbon-Aware Resource Management
CNN	Convolutional Neural Network
DNN	Deep Neural Network
GPU	Graphics Processing Unit
NDC	Nationally Determined Contribution
PPC	Performance per Carbon

PPW	Performance per Watt
SLA	Service-Level Agreement
TDP	Thermal Device Power
TPU	Tensor Processing Unit

References

1. Li, B.; Arora , R.; Samsi, S.; Patel, T.; Arcand, W.; Bestor, D.; Byun, C.; Roy , R.B.; Bergeron, B.; Holodnak, J.; et al. Ai-enabling workloads on large-scale gpu-accelerated system: Characterization, opportunities, and implications. In Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Seoul, Republic of Korea, 2–6 April 2022; pp. 1224–1237.
2. NVIDIA RTX Blackwell GPU Architecture. Available online: https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf (accessed on 29 November 2025).
3. Intel Xeon Platinum 8480+ Processor. Available online: https://www.intel.com/content/www/us/en/products/sku/231746/intel-xeon-platinum-8480-processor-105m-cache-2-00-ghz/specifications.html (accessed on 29 November 2025).
4. AMD EPYC 9654 Processor. Available online: https://www.amd.com/en/products/processors/server/epyc/4th-generation-9004-and-8004-series/amd-epyc-9654.html (accessed on 29 November 2025).
5. IEA Energy and AI. 2025. Available online: https://www.iea.org/reports/energy-and-ai (accessed on 29 November 2025).
6. IEA Data Centre Energy Use: Critical Review of Models and Results. 2025. Available online: https://www.iea-4e.org/wp-content/uploads/2025/05/Data-Centre-Energy-Use-Critical-Review-of-Models-and-Results.pdf (accessed on 29 November 2025).
7. Ember Global Electricity Review. 2024. Available online: https://ember-energy.org/app/uploads/2024/05/Report-Global-Electricity-Review-2024.pdf (accessed on 29 November 2025).
8. IPCC Climate Change 2023 Synthesis Report. Available online: https://www.ipcc.ch/report/ar6/syr/downloads/report/IPCC_AR6_SYR_SPM.pdf (accessed on 29 November 2025).
9. UNFCCC Paris Agreement 2015. Available online: https://unfccc.int/files/meetings/paris_nov_2015/application/pdf/paris_agreement_english_.pdf (accessed on 29 November 2025).
10. Jouppi, N.; Kurian, G.; Li, S.; Ma, P.; Nagarajan, R.; Nai, L.; Patil, N.; Subramanian, S.; Swing, A.; Towles, B.; et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA), Orlando, FL, USA, 17–21 June 2023; pp. 1–14.
11. FuriosaAI. Available online: https://www.furiosa.ai/ (accessed on 29 November 2025).
12. Shaped.ai. Available online: https://www.shaped.ai/ (accessed on 29 November 2025).
13. Wang, S.; Chen, S.; Shi, Y. Utilization-prediction-aware energy optimization approach for heterogeneous gpu clusters. *J. Supercomput.* **2024**, *80*, 9554–9578. [CrossRef]
14. Xu, K.; Sun, D.; Tian, H.; Zhang, J.; Chen, K. Carbon-efficient Resource Scheduling for Machine Learning Clusters. In Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25), Philadelphia, PA, USA, 28–30 April 2025; pp. 999–1014.
15. Jouppi, N.P.; Yoon, D.H.; Ashcraft, M.; Gottscho, M.; Jablin, T.B.; Kurian, G.; Laudon, J.; Li, S.; Ma, P.; Ma, X.; et al. Ten lessons from three generations shaped google's tpuv4i: Industrial product. In Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA), Online Event, 14–19 June 2021; pp. 1–14.
16. Blalock, D.; Ortiz, J.J.G.; Frankle, J.; Guttag, J. What is the State of Neural Network Pruning? In Proceedings of the Machine Learning and Systems (MLSyS) 2020, Austin, TX, USA, 2–4 March 2020; pp. 129–146.
17. Frantar, E.; Alistarh, D. SparseGPT. massive language models can be accurately pruned in one-shot. In Proceedings of the 40th International Conference on Machine Learning (ICML), Hawaii, HI, USA, 23–29 July 2023; pp. 10323–10337.
18. Nugent, D.; Sovacool, B.K. Assessing the lifecycle greenhouse gas emissions from solar PV and wind energy: A critical meta-survey. *Energy Policy* **2014**, *65*, 229–244. [CrossRef]
19. Wang, Q.; Li, L.; Luo, W.; Zhang, Y.; Wang, B. DSO: A GPU energy efficiency optimizer by fusing dynamic and static information. In Proceedings of the IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS) 2024, Guangzhou, China, 19–21 June 2024; pp. 1–6.
20. Hong, S.P.; Kim, H.S. An integrated GPU power and performance model. In Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA), Saint-Malo, France, 19–23 June 2010; pp. 280–289.
21. Kang, D.K.; Lee, K.B.; Kim, Y.C. Cost efficient gpu cluster management for training and inference of deep learning. *Energies* **2022**, *15*, 474. [CrossRef]
22. He, H.; Hong, S. Minimizing the operation cost of distributed green data centers with energy storage under carbon capping. *J. Comput. Syst. Sci.* **2021**, *118*, 28–52. [CrossRef]

23. Lin, M.; Wierman, A.; Andrew, L.L.; Thereska, E. Dynamic right-sizing for power-proportional data centers. *IEEE/ACM Trans. Netw.* **2012**, *21*, 1378–1391. [CrossRef]
24. Park, H.G.; Kang, D.K. Renewable-aware frequency scaling approach for energy-efficient deep learning clusters. *Appl. Sci.* **2024**, *14*, 776. [CrossRef]
25. Kang, D.K.; Ha, Y.G.; Peng, L.; Youn, C.H. Cooperative Distributed GPU Power Capping for Deep Learning Clusters. *IEEE Trans. Ind. Electron.* **2022**, *69*, 7244–7254. [CrossRef]
26. Guan, H.; Yao, J.; Qi, Z.; Wang, R. Energy-efficient SLA guarantees for virtualized GPU in cloud gaming. *IEEE Trans. Parallel Distrib. Syst.* **2014**, *26*, 2434–2443. [CrossRef]
27. Kang, D.K.; Yang, E.J.; Youn, C.H. Deep learning-based sustainable data center energy cost minimization with temporal MACRO/MICRO scale management. *IEEE Access* **2018**, *7*, 5477–5491. [CrossRef]
28. Pytorch-cifar100. Available online: https://github.com/weiaicunzai/pytorch-cifar100 (accessed on 29 November 2025).
29. NVIDIA NVML. Available online: https://developer.nvidia.com/management-library-nvml (accessed on 29 November 2025).
30. Simonyan, K.; Zisserman, A. Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv* **2014**, arXiv:1409.1556. [CrossRef]
31. Szegedy, C.; Ioffe, S.; Vanhoucke, V.; Alemi, A.A. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. In Proceedings of the 31th AAAI Conference on Artificial Intelligence (AAAI-17), San Francisco, CA, USA, 4–9 February 2017.
32. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep Residual Learning for Image Recognition. In Proceedings of the 29th IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778.
33. Huang, G.; Sun, Y.; Liu, Z.; Sedra, D.; Weinberger, K.Q. Deep Networks with Stochastic Depth. In Proceedings of the 14th European Conference on Computer Vision (ECCV), Amsterdam, The Netherlands, 11–14 October 2016; pp. 646–661.
34. Measurement and Instrumentation Data Center (MIDC). Available online: https://midcdmz.nrel.gov/ (accessed on 29 November 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.